Seminar 04 - REST API, OpenAPI, Kubb, TanStackQuery

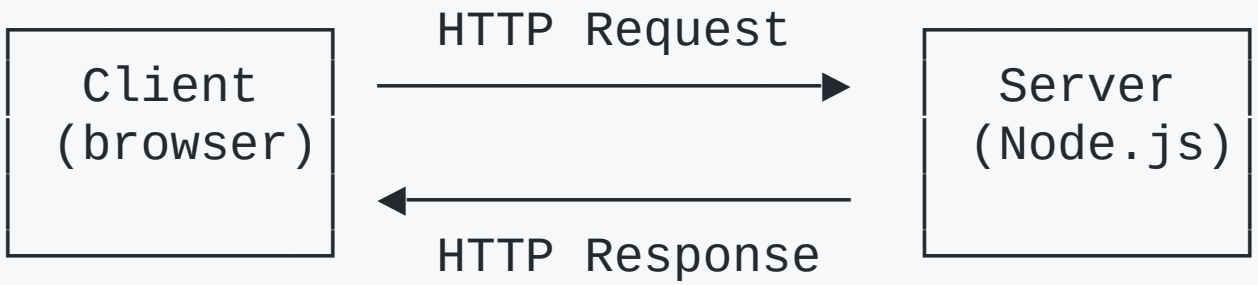
PB138 - Introduction to Web Development

Agenda

1. **REST API** - introduction, best practices
2. **OpenAPI specification**
3. **Kubb** - code generation from specs
4. **TanStack Query**

Client - server

The **client** (browser, mobile app…) sends a *request* to a **server**, which processes it and sends back a *response*.



- **Backend** - handles server side
- **Frontend** - handles client side
- The server exposes **endpoints** (URLs) the client can call

Why?

- **Separation of concerns** - UI logic =/= business logic
- **Scalability** - scale frontend and backend independently
- **Multiple clients** - web, mobile, IoT can share the same API
- **Security** - sensitive data and secrets stay on the server
- **Team autonomy** - frontend and backend teams work in parallel

What Is a REST API?

REpresentational **S**tate **T**ransfer

- API as the interface/contract between the client and the server
- **Resources** - everything is a resource identified by a URL, e.g. `/products/42`
- **Stateless** - each request carries all info the server needs
- **Uniform interface** - standard HTTP methods and status codes

HTTP Methods

Method	Purpose
`GET`	Read a resource
`POST`	Create a new resource
`PUT`	Full replace of a resource
`PATCH`	Partial update
`DELETE`	Remove a resource

HTTP Methods

Examples (with their associated endpoints)

```
GET    /products          → list all products
GET    /products/:id      → get one product by ID
POST   /products          → create a new product
PUT    /products/:id      → fully replace a product
PATCH  /products/:id      → partially update a product
DELETE /products/:id      → delete a product
```

- Use **nouns** for resources, not verbs (`/products`, not `/getProducts`)
- Use **plural** names
- Nest for relationships: `/users/:userId/orders`

HTTP Status Codes

Range	Category	Meaning
1xx	Informational	Request received, continuing…
2xx	Success	Request succeeded
3xx	Redirection	Further action needed
4xx	Client Error	Problem with the request
5xx	Server Error	Server failed to fulfill a valid request

CORS - Cross-Origin Resource Sharing

Your frontend runs on `http://localhost:5173`.
Your API runs on `http://localhost:3000`.

These are **different origins** → the browser blocks the request by default.

```
Origin = scheme + host + port
http://localhost:5173  ≠  http://localhost:3000
```

CORS is a mechanism that lets the **server** tell the browser to trust requests from an origin.

Best practices and tips

- Use **JSON** for request and response bodies
- Use **plural nouns** for resource names (`/products`, not `/product`)
- Return **appropriate status codes** - don't send `200` for everything
- API versioning: `/api/v1/products`
- Validate input on the server (*never* trust the client)
- Use **pagination** for list endpoints (`?page=1&limit=20`)
- Return useful **error messages** with a consistent shape
- **Log** requests and errors

Task 1

Your first task is to finish the implmementation of API endpoints in `server/src/index.ts`

You get a skeleton solution of the server. Your task is to finish the implementation of the incomplete endpoints:

`/products:id` - GET endpoint that returns a singular product identified by its id.
`/products` - POST endpoint that should use zod to validate the body of the request and create a product.

OpenAPI Specification

OpenAPI (formerly Swagger) is a **standard format** for describing REST APIs.

- Written in **YAML** or **JSON**
- Describes endpoints, methods, request/response bodies, parameters, status codes
- Acts as a **single source of truth**
- **Contract-first** (or **API-first**) development

Why?

Benefit	Details
Documentation	Auto-generate interactive docs (Swagger UI, Redoc)
Code generation	Generate TypeScript types, API clients, server stubs
Validation	Auto-validate requests/responses against the schema
Team communication	Frontend & backend agree on the contract *before* coding
Testing	Generate mock servers and contract tests

OpenAPI - Basic Structure

```yaml
openapi: 3.1.0
info:
  title: Example API
  version: 1.0.0
  description: This document describes my API

servers:
  - url: http://localhost:3000
    description: Local development

paths:
  /products:
    get:
      summary: List all products
      responses:
        "200":
          description: A list of products
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/Product"
```

OpenAPI - Defining Schemas

```yaml
components:
  schemas:
    Product:
      type: object
      required: [id, title, price]
      properties:
        id:
          type: integer
          example: 1
        title:
          type: string
          example: "Laptop"
        price:
          type: number
          format: float
          example: 999.99
```

OpenAPI - POST example

```yaml
paths:
  /products:
    post:
      summary: Create a new product
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/CreateProductBody"
      responses:
        "201":
          description: Product created
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Product"
        "400":
          description: Validation error
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
```

Tips

- Start with **schemas first**
- Document **all possible responses**
- Add `example` values
- Use `operationId`, useful for generating function names
- **Version control** the spec

Kubb

Kubb (https://kubb.dev) is a **code generator** that can take a spec and generate:

- **TypeScript types / models** - fully typed `Product`, `CreateProductBody`, etc.
- **API client functions** - ready-to-use `fetch`-based functions
- **Zod schemas** - runtime validation that matches your spec

Setup

```
npx kubb init
npm install --save-dev @kubb/plugin-zod
```

Create `kubb.config.ts`:

```ts
import { defineConfig } from "@kubb/core";
import { pluginTs } from "@kubb/plugin-ts";
import { pluginClient } from "@kubb/plugin-client";

export default defineConfig({
  input: { path: "./openapi.yaml" },
  output: { path: "./src/gen" },
  plugins: [
    pluginTs(),
    pluginClient()
  ],
});
```

Generate code

After `npx kubb generate`, you get code ready to use directly:

```
src/gen/
├── models/                ← TypeScript types
│   ├── Product.ts
│   └── CreateProductBody.ts
├── clients/               ← API client functions
│   └── productsClient.ts
└── query/
    └── useListProducts.ts ← TanStack queries
```

Task 02

After helping finish the backend, you will need to finish the API specification accordingly, located in the
root folder in `openapi.yaml`

Then `cd` into `client` folder and use `npx kubb generate` to generate Kubb types.

TanStack Query

TanStack Query (https://tanstack.com/query) (formerly React Query) manages **server state** in your frontend.

Features:

- **Automatic caching** and background re-fetching
- Built-in `isLoading`, `isError`, `data` states
- **Mutations** with optimistic updates & cache invalidation
- Devtools for inspecting cache state

TanStack Query

Solved problems:

- Manually tracking `loading` / `error` / `data` with `useEffect` + `useState`
- Forgetting to handle race conditions, stale data, refetching
- Building your own caching layer

Caching

- With TanStack Query, components can reuse already cached data
- Cache is keyed by `queryKey`
- Data is taken from cache if present, fetched otherwise

Key	Value
`["products"]`	`[{ id: 1, … }, { id: 2, … }, …]`
`["products", 42]`	`{ id: 42, name: "USB-C Hub", … }`

Caching

- Data changes on should lead to re-fetch
- Done by calling `invalidateQueries`, e.g. `invalidateQueries(["products"])`
- Cache entries are then marked as stale and will be re-fetched (and cached again)

Setup

```
npm install @tanstack/react-query
```

Wrap the app:

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <ProductList />
    </QueryClientProvider>
  );
}
```

useQuery - Fetching Data

```
import { useQuery } from "@tanstack/react-query";

function ProductList() {
  const { data, isLoading, isError, error } = useQuery({
    queryKey: ["products"],
    queryFn: () =>
      fetch("http://localhost:3000/products").then((res) => res.json()),
  });

  if (isLoading) return <p>Loading…</p>;
  if (isError)   return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map((p) => (
        <li key={p.id}>{p.title} - {p.price} €</li>
      ))}
    </ul>
  );
}
```

useMutation - Creating Data

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

function AddProductForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newProduct) =>
      fetch("http://localhost:3000/products", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(newProduct),
      }).then((res) => res.json()),

    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["products"] });
    },
  });
}
```

Task 03

The last task is to finish the client implementation in `client/src/ProductPage.tsx` using TanStack Query.
Use the code generated by Kubb to implement:

- Query for obtaining a list of Products
- Mutation for creating a new Product
- Mutation for deleting a Product

For proper testing, start the development server in by calling `npm run dev` in the `server` folder.
The server runs on port 3000 by default and the demo client expects this to be the case.

Recap

Topic	Key Takeaway
Client–Server	Clean separation; API is the contract
REST	Resources + HTTP methods + status codes
CORS	Server must explicitly allow cross-origin requests
OpenAPI	Description of the contract, single source of truth
Kubb	Spec → generation of types, client, hooks
TanStack Query	Server state management with caching and mutations

Next week

A look at databases, ERD modeling and object-relational mappers